Working Time 🤓

L3: Preprocesamiento y escalamiento de datos

Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

   Aplicar técnicas de preprocesamiento y escalamiento de datos utilizando Python y Scikit-Learn.

   Consigna

   Utilizando el siguiente conjunto de datos ficticio sobre clientes, realiza los siguientes pasos:

ID	*Edad*	*Ciudad*	*Salario*
1	*25*	*Madrid*	*30000*
2	*45*	*Sevilla*	*50000*
3	*30*	*Madrid*	*NaN*
4	*40*	*Barcelona*	*40000*

   ***Imputar** el valor faltante en la columna **Salario** utilizando la media.*
   *Transformar la columna **Ciudad** en variables categóricas aplicando:*

   - ***Label Encoding***

- ***One-Hot Encoding***
- ***Variables Dummy***

Aplicar técnicas de escalamiento sobre las columnas ***Edad*** *y* ***Salario*** *utilizando:*

- ***Normalización Min-Max***
- ***Estandarización (Z-Score)***

Mostrar los datos resultantes después de cada etapa para observar los cambios.

2. **¿Dónde se lleva a cabo?** 👩‍💻

   Visual Studio Code

3. **Tiempo de dedicación** ⏳

   1 hora

4. **Recursos**🛠️

   - https://scikit-learn.org/stable/modules/preprocessing.html
   - https://www.analyticsvidhya.com/blog/2020/04/feature-scaling-machine-learning-normalization-standardization/

5. **Plus** ➕

   ✅ *Probar el uso de* ***KNNImputer*** *para imputar el salario en lugar de la media.*

   ✅ *Aplicar escalamiento solo a las columnas numéricas utilizando un* ***ColumnTransformer****.*

6. ⚠️**Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler, StandardScaler

from sklearn.impute import SimpleImputer


# Datos

data = {

    'ID': [1, 2, 3, 4],

    'Edad': [25, 45, 30, 40],

    'Ciudad': ['Madrid', 'Sevilla', 'Madrid', 'Barcelona'],

    'Salario': [30000, 50000, np.nan, 40000]

}


df = pd.DataFrame(data)

print("Datos originales:\n", df)


# 1. Imputar salario

imputer = SimpleImputer(strategy='mean')

df['Salario'] = imputer.fit_transform(df[['Salario']])

print("\nDatos con salario imputado:\n", df)


# 2. Label Encoding

le = LabelEncoder()
```

```
df['Ciudad_Label'] = le.fit_transform(df['Ciudad'])

print("\nLabel Encoding:\n", df[['Ciudad', 'Ciudad_Label']])


# 3. One-Hot Encoding

ohe = OneHotEncoder(sparse=False)

ciudad_encoded = ohe.fit_transform(df[['Ciudad']])

df_ohe = pd.DataFrame(ciudad_encoded,
columns=ohe.get_feature_names_out(['Ciudad']))

print("\nOne-Hot Encoding:\n", df_ohe)


# 4. Variables Dummy

df_dummy = pd.get_dummies(df['Ciudad'], drop_first=True)

print("\nVariables Dummy:\n", df_dummy)


# 5. Min-Max Scaling

scaler_minmax = MinMaxScaler()

df[['Edad_MinMax', 'Salario_MinMax']] =
scaler_minmax.fit_transform(df[['Edad', 'Salario']])

print("\nMin-Max Scaling:\n", df[['Edad_MinMax', 'Salario_MinMax']])


# 6. Estandarización

scaler_std = StandardScaler()

df[['Edad_Std', 'Salario_Std']] = scaler_std.fit_transform(df[['Edad',
'Salario']])

print("\nEstandarización:\n", df[['Edad_Std', 'Salario_Std']])
```